Trajecto: AI-Enhanced 3D Pen Tracking

Technical Specification System Architecture

Trajecto Development Team

January 26, 2026

Contents

1 System Abstract

1.1 Goal

The primary objective of Trajecto is to achieve **sub-centimeter accuracy** in 3D handwriting reconstruction using low-cost inertial sensors. Currently, the system focuses on scale stabilization and reconstructing macro trajectories to bridge the gap between expensive motion capture systems and consumer electronics.

1.2 Key Approach

The system employs a **Closed-Loop Hybrid Architecture** that fuses physics-based estimation with deep learning corrections. Instead of replacing the physical model, a Temporal Convolutional Network (TCN) acts as a "virtual sensor," correcting the drift inherent in the double integration of noisy IMU data while preserving the physical consistency guaranteed by an Error-State Kalman Filter (ESKF).

1.3 Target Platform

The system is optimized for resource-constrained embedded platforms, specifically targeting the **Espressif ESP32-C3** (RISC-V 32-bit Single Core @ 160MHz) running at an inference rate of 50Hz and sensor integration rate of 400Hz.

2 Mathematics and AI Architecture

2.1 Error-State Kalman Filter (ESKF)

The backbone of the trajectory estimation is an Error-State Kalman Filter (ESKF) which separates the state into a nominal state (x) and an error state (δx).

$$x_{true} = x_{nominal} \oplus \delta x \tag{1}$$

The 15-dimensional error state vector is defined as:

$$\delta x = [\delta p, \delta v, \delta \theta, \delta b_g, \delta b_a]^T \tag{2}$$

where $\delta p, \delta v$ are position/velocity errors, $\delta \theta$ is the orientation error (small angle), and $\delta b_g, \delta b_a$ are bias errors.

2.2 Parallel Associative Scan

To enable efficient training over long sequences (thousands of timesteps), the sequential Kalman Filter operations ($O(N)$) are reformulated as an **associative scan (prefix sum)** problem ($O(\log N)$ on GPU).

The covariance propagation $P_{t+1} = F_t P_t F_t^T + Q_t$ is transformed into an associative operator $\otimes$:

$$(F_2, Q_2) \otimes (F_1, Q_1) = (F_2 F_1, \quad F_2 Q_1 F_2^T + Q_2) \tag{3}$$

This allows gradients to flow from the position loss at the end of a trajectory back to the neural network weights at the beginning, solving the "vanishing gradient" problem in long-term estimation.

2.3 Temporal Convolutional Network (TCN)

The TCN functions as a hyper-sensor, observing raw IMU data to predict corrections. It features a specific **Y-Branch Architecture**:

- **Backbone**: 2 layers of shared causal 1D convolutions for feature extraction.

- **Dynamic Branch**: Small receptive field (dilations 1, 2) to capture fast motion dynamics. Predicts **Velocity Residuals** (δv) and **Covariance** (R).
- **Static Branch**: Large receptive field (dilations 4, 8) to detect stationarity. Predicts **ZUPT Probability** and **Gravity Alignment**.

2.3.1 SymLog Activation

For velocity corrections, the network uses a **Signed Log-1p** activation to handle high dynamic range signals without saturation or gradient vanishing:

$$y = \text{sign}(x) \cdot \ln(1 + |x|) \cdot \sigma \tag{4}$$

This allows precise corrections near zero while compressing large velocity spikes.

3 Data Engineering Sim-to-Real

3.1 Julia-Based Physics Simulator

A high-fidelity simulator written in Julia generates synthetic training data to bootstrap the model ("Sim2Real").

- **Kinematics**: Models human handwriting using inverse kinematics and sigma-lognormal velocity profiles.
- **Noise Modeling**: Injects realistic sensor noise based on Allan Variance characteristics of the BMI270.
- **Grip Modeling**: Simulates biomechanical constraints of a pen held in a hand.

3.2 iPad-Based 6-DOF Data Acquisition

To obtain ground truth, we developed a custom iPadOS application ("Trajectory Recorder") that leverages the Apple Pencil Pro.

- **High Frequency**: Captures coalesced touch events at **240Hz**+.
- **6-DOF Tracking**: Records (x, y, z_{hover}) position alongside Azimuth, Altitude, and Roll angles.
- **Protocol**: Enforces a "Tap-Wait-Write" protocol to synchronize the independent clocks of the ESP32 and iPad via acceleration spikes.

4 Hardware Firmware Implementation

4.1 Hardware Design

The custom PCB is engineered for signal integrity in a stylus form factor (13mm $\times$ 60mm).

- **Layer Stack**: 4-layer PCB with dedicated power plane segmentation to separate digital switching noise from sensitive analog sensors.
- **Isolation**: A 600Ω ferrite bead isolates the 3.3V analog rail for the IMU and FSR.
- **Sensors**:
  - **IMU**: Bosch BMI270 (16-bit, Low-noise) sampled at 400Hz.
  - **FSR**: Force Sensitive Resistor with active Op-Amp conditioning for reliable contact detection.

4.2 Embedded Optimization

The firmware implements the full inference pipeline on the ESP32-C3:

- **TFLite Micro**: Runs the TCN model quantized to **INT8** for memory and speed efficiency.
- **Stateful Inference**: The TCN buffer is managed statefully (ring buffer) to allow continuous inference on streaming data without re-computing the full receptive field.
- **Stride Optimization**: Inference runs at a strided rate (e.g., 50Hz) while the physics filter integrates at the full sensor rate (400Hz) to balance compute load and latency.